Guia minucioso do código — Acervo

Este guia acompanha a implementação real em src/main e src/test. Seu objetivo é permitir que o estudante compreenda a responsabilidade de cada arquivo, o motivo das principais decisões e a sequência de execução. O relatório técnico apresenta o contexto acadêmico; este documento aprofunda o funcionamento interno.

1. Como ler o projeto

Comece por model/ItemMidia.java, depois dao/ItemMidiaDao.java e service/ItemMidiaService.java. Em seguida leia dao/JdbcItemMidiaDao.java e config. Por último, acompanhe web/ItemMidiaServlet.java e as JSPs. Essa ordem revela primeiro o problema e as regras, depois os mecanismos HTTP e SQL.

A chamada normal percorre navegador → filtro → Servlet → serviço → DAO → JDBC → banco. O retorno percorre banco → DAO → serviço → Servlet → JSP → navegador. Nem toda requisição consulta o banco: abrir um formulário vazio apenas prepara os atributos da página.

2. Organização e dependências de construção

2.1 pom.xml

O elemento packaging define war, formato convencional de aplicação web Java. O finalName catalogo determina target/catalogo.war e a pasta explodida target/catalogo. A propriedade maven.compiler.release=17 informa a versão de linguagem e APIs Java que o compilador pode utilizar; o texto-fonte é UTF-8.

As dependências Servlet, Tomcat, JSTL, H2 e JUnit correspondem a funções distintas. Servlet fornece o contrato HTTP; Tomcat executa e compila as páginas; JSTL oferece tags de apresentação; H2 implementa o banco; JUnit executa os testes. A API Servlet tem escopo provided, pois o contêiner já a disponibiliza. JUnit tem escopo test e não acompanha a aplicação.

O Compiler compila o código. Surefire encontra classes *Test. War empacota a aplicação e exclui tomcat-embed e ecj, necessários somente ao inicializador embutido. Dependency copia as dependências de execução para target/dependency. Failsafe encontra *IT e valida os testes HTTP após a preparação da aplicação web. Dessa forma, a suíte HTTP pode usar JSPs, classes e bibliotecas já reunidas em target/catalogo.

2.2 maven.ps1

O script usa a pasta do próprio projeto como diretório de trabalho. Procura mvn.cmd no PATH e, se não existir, baixa a distribuição 3.9.9 para .tools. Compara o SHA-512 com o valor publicado no repositório antes de extrair e executar. A primeira execução necessita de conexão à internet. O repositório Maven local é configurado em .m2, evitando depender de alterações na configuração global da máquina.

Os argumentos recebidos são repassados ao Maven. O código de saída é propagado para que iniciar.ps1 interrompa o processo se a compilação falhar. O finally restaura o diretório do terminal.

A opção ExecutionPolicy Bypass usada nos exemplos vale para o processo iniciado, sem instruir uma mudança permanente da política do computador.

2.3 iniciar.ps1

Verifica a existência do Java, executa package e monta o classpath com target/classes e target/dependency/*. O parâmetro -SemCompilar permite reutilizar uma compilação existente. O diretório de trabalho é relevante porque a URL padrão H2 e o caminho do webapp são relativos a ele. O servidor roda no mesmo terminal e pode ser encerrado com Ctrl+C.

3. Entidade e enum

3.1 ItemMidia.java

A classe representa uma obra. O ID é Long porque um item novo ainda não tem identificador: null representa o estado anterior à inserção. Título, autor/diretor, gênero e sinopse são String; ano é int; tipo é TipoMidia.

Todos os atributos são private final. O modificador private impede acesso direto e final impede troca da referência ou do valor após a construção. Isso não significa que final tornaria qualquer objeto profundamente imutável; aqui, os próprios tipos usados são imutáveis ou primitivos. O objeto pode ser compartilhado com segurança sem receber mutações de um setter.

O construtor inicializa o estado e impede referências nulas nos textos e no tipo com Objects.requireNonNull. Ele não substitui a validação de negócio: tamanhos, valores vazios e calendário são responsabilidade do serviço. O DAO reconstrói a entidade a partir dos registros já armazenados.

comId cria outra entidade com os mesmos dados e o ID recebido. O INSERT pode retornar a versão persistida sem alterar a instância fornecida pelo chamador. Os getters servem ao serviço, DAO e Expression Language das JSPs: item.titulo resolve getTitulo, e item.tipoMidia.descricao resolve getTipoMidia().getDescricao().

3.2 TipoMidia.java

LIVRO, FILME e SERIE são identificadores estáveis, gravados no banco com name(). Cada constante guarda uma descrição amigável: Livro, Filme e Série. O construtor do enum associa as duas representações, e getDescricao oferece o texto para a interface. Essa separação evita depender de acentos no valor persistido ou de valores livres para as categorias.

Não há subclasses Livro e Filme porque o comportamento atual é igual. A POO não exige criar herança em toda diferença de rótulo. Se um futuro requisito trouxer comportamentos próprios, o modelo poderá evoluir com base nessa necessidade.

4. Exceções e divisão das falhas

4.1 ValidacaoException.java

Recebe um mapa de chave de campo para mensagem. A ordem é conservada por LinkedHashMap e o conteúdo é exposto por uma visão não modificável de uma cópia. Assim, o consumidor não altera o conjunto de erros depois da emissão. O controlador coloca esse mapa em request.erros e a JSP apresenta cada mensagem ao lado do campo correspondente.

4.2 NaoEncontradoException.java

Representa o caso de uma obra não existir. O serviço a utiliza quando uma consulta retorna Optional.empty ou uma escrita afeta zero linhas. O controlador traduz para 404. Essa classe mantém o significado de negócio separado de SQLException e de códigos HTTP.

4.3 PersistenciaException.java

Encapsula falhas de armazenamento e preserva a causa com super(mensagem, causa). Essa causa é importante para depuração no log. O controlador apresenta uma mensagem pública genérica, sem reutilizar o texto detalhado da exceção SQL. O serviço e o controlador não precisam declarar SQLException em suas assinaturas.

5. Configuração e inicialização do banco

5.1 FabricaConexao.java

É uma interface funcional com abrir(), que retorna Connection e pode lançar SQLException. O DAO conhece esse contrato, mas não decide a URL, as credenciais nem o ambiente de execução. Nos testes, uma expressão lambda pode devolver uma conexão de teste ou lançar uma falha simulada.

5.2 ConexaoJdbc.java

O construtor armazena URL, usuário e senha e verifica a presença do driver H2. doAmbiente lê CATALOGO_DB_URL, CATALOGO_DB_USER e CATALOGO_DB_PASSWORD, utilizando os padrões documentados quando as variáveis não existem. A senha vazia pode ser uma escolha explícita para o banco embutido da demonstração.

abrir chama DriverManager.getConnection. Cada operação DAO recebe uma nova conexão e a fecha ao terminar. Não existe uma Connection global compartilhada entre requisições. Essa decisão simplifica o ciclo de vida e evita compartilhar um objeto de conexão simultaneamente. Um pool de conexões poderia ser introduzido atrás da interface em outro contexto.

5.3 BancoDados.java

inicializar lê /db/schema.sql do classpath com UTF-8. A ausência do recurso é uma falha de empacotamento e impede uma inicialização silenciosa e incompleta. O script é dividido por ponto e vírgula e seus comandos são executados. Esse leitor é adequado ao esquema simples entregue, não é um parser SQL geral para funções ou rotinas com ponto e vírgula interno.

O Statement simples recebe somente o SQL estático do projeto. Não lê SQL de parâmetros HTTP. Os recursos de arquivo, conexão e comando são fechados por try-with-resources. Falhas de SQL ou leitura são encapsuladas como persistência. A criação usa IF NOT EXISTS e nunca apaga dados ou adiciona obras automaticamente.

5.4 schema.sql

GENERATED BY DEFAULT AS IDENTITY cria IDs no banco e PRIMARY KEY garante identidade única. NOT NULL impede valores ausentes. CHECK de comprimento impede textos vazios ou compostos apenas de espaços comuns. VARCHAR limita o tamanho das colunas. A restrição do tipo impede valores fora do enum.

A regra estática do banco para o ano é 1 a 9999. O serviço aplica uma regra mais específica, até o próximo ano do calendário. Essa separação evita uma migração anual do esquema. Há defesa de integridade no banco e mensagens orientadas ao usuário no serviço. A aplicação não expõe escrita direta ao banco pelo navegador.

6. Contrato DAO

ItemMidiaDao declara cinco operações. inserir retorna o item com ID, buscar retorna uma lista, buscarPorId retorna Optional, atualizar e excluir retornam boolean para indicar se houve uma linha afetada. Esses retornos deixam explícita a diferença entre sucesso, conjunto vazio e registro inexistente.

A interface não inclui request, response, JSP, PreparedStatement ou SQLException. O contrato fala em itens e resultados do catálogo. A implementação pode variar sem transferir detalhes JDBC ao serviço. O contrato pressupõe que a camada de serviço já normalizou o termo de busca e validou os dados da escrita.

7. JdbcItemMidiaDao, método a método

7.1 Estado e constantes

O DAO recebe FabricaConexao no construtor e a conserva em atributo final. COLUNAS concentra a projeção comum das consultas. Não usa SELECT *: a lista explícita de colunas mostra quais campos o mapeamento espera. Essa constante é interna, sem dados do usuário.

7.2 inserir

Monta INSERT com seis placeholders. Abre conexão e PreparedStatement configurado com RETURN_GENERATED_KEYS. preencher atribui os valores de domínio aos seis parâmetros. executeUpdate executa a escrita e getGeneratedKeys obtém o ID gerado pelo banco. Se não houver chave retornada, a operação não inventa um ID; sinaliza SQLException, encapsulada na saída do método.

ResultSet, comando e conexão têm escopos de fechamento próprios. O retorno é item.comId, conservando a entidade original. A escrita atual é uma única instrução e usa o autocommit padrão. Não existe uma sequência de várias alterações que exigiria begin/commit manual neste caso.

7.3 buscar

Quando o termo é vazio, executa SELECT com ORDER BY LOWER(titulo), id. Quando há termo, cria o padrão com % nas extremidades e aplica LOWER a título e autor. A condição usa OR, portanto qualquer um dos dois campos pode encontrar a obra.

O caractere ! é o escape da expressão LIKE. Primeiro ! vira !!, depois % vira !% e _ vira !_. A ordem importa: se ! fosse escapado por último, os escapes introduzidos seriam novamente alterados. O padrão é passado por setString aos dois placeholders. A parte constante do SQL pode ser concatenada com outra constante sem criar SQL Injection; o ponto essencial é não concatenar entrada do usuário ao comando.

A busca é insensível à caixa, mas não remove acentos. A ordenação é simples, não foi configurada uma collation linguística portuguesa. LIKE com % no início pode exigir varredura das linhas e não é anunciado como otimizado para grandes acervos.

7.4 buscarPorId

Reutiliza consultar com WHERE id = ?. O lambda configura setLong. O resultado se transforma em stream().findFirst(). Como ID é chave primária, há no máximo uma linha. Se não houver linha, o retorno é Optional.empty, e o serviço decide como essa ausência afeta o caso de uso.

7.5 atualizar

Executa UPDATE dos seis atributos da obra com WHERE id = ?. Reutiliza preencher para os parâmetros 1 a 6 e adiciona ID na posição 7. O boolean retornado informa se o registro existia no momento da escrita. Não depende de uma consulta prévia sujeita ao intervalo entre leitura e alteração.

7.6 excluir

Executa DELETE com um único parâmetro long. Não recebe um fragmento SQL ou uma lista de IDs em texto livre. Retorna false se nenhuma linha existia. A confirmação visual e a proteção do método HTTP pertencem à interface e ao filtro, não ao DAO.

7.7 consultar

Centraliza a estrutura de consultas: abre recursos, recebe um configurador de parâmetros, executa SELECT, percorre resultado.next, chama mapear e retorna ArrayList. O while transforma cada linha em uma entidade. Se não há linhas, a lista fica vazia. Qualquer SQLException é encapsulada com indicação da operação.

7.8 modificar

Centraliza a estrutura das escritas de atualização e exclusão: abre recursos, aplica os parâmetros e verifica executeUpdate() > 0. Não é usado pelo INSERT porque a inserção precisa retornar chaves geradas. Essa diferença preserva uma abstração simples, sem forçar todas as operações a caberem em uma função genérica difícil de entender.

7.9 preencher e mapear

preencher estabelece a relação fixa entre parâmetros 1–6 e os campos de ItemMidia. INSERT e UPDATE usam a mesma ordem. mapear realiza a transformação inversa, lendo as colunas por nome e convertendo tipo_midia por TipoMidia.valueOf. Essa centralização impede que consultas diferentes reconstruam a entidade de formas divergentes.

7.10 Interface privada Parametros

O método aplicar recebe PreparedStatement e admite SQLException. Assim, os lambdas de configuração podem utilizar os setters JDBC sem tratar a mesma exceção em todos os chamadores. A interface é privada porque é um detalhe interno da implementação DAO, não uma API de negócio.

8. ItemMidiaService, método a método

8.1 Construtores e relógio

O construtor simples utiliza Clock.systemDefaultZone. O construtor completo recebe DAO e Clock. Ambos os objetos são obrigatórios. Clock permite testar o limite de ano com uma data fixa, sem o teste mudar de resultado quando o calendário muda. O serviço não instancia JdbcItemMidiaDao internamente.

8.2 salvar

Recebe ID opcional e mapa de campos textuais. Cria um mapa de erros e usa texto para validar título, autor/diretor, gênero e sinopse. Cada valor é normalizado com strip, removendo espaços das extremidades. O ano é convertido por Integer.parseInt e comparado com 1 e Year.now(relogio)+1. Valor não numérico ou fora de int gera mensagem específica.

O tipo é convertido com TipoMidia.valueOf. Uma categoria não prevista acumula erro. Todas as validações são avaliadas antes de qualquer escrita, e o método lança uma única ValidacaoException com o conjunto. Isso permite apresentar vários erros de uma vez, em lugar de exigir um envio por campo.

Com dados válidos, constrói ItemMidia. ID nulo seleciona inserir; ID presente seleciona atualizar. Se atualizar retorna false, o serviço lança NaoEncontradoException. O método devolve a entidade persistida ou atualizada. A distinção entre cadastro e edição é compartilhada, evitando repetir as regras em dois serviços.

8.3 buscar

Normaliza null como termo vazio, remove espaços e limita o tamanho a 255 caracteres. O DAO recebe um termo consistente. Um termo longo gera ValidacaoException associada ao campo q. Como se trata de uma requisição GET, o controlador apresenta uma página de erro 400 em vez do formulário de obra.

8.4 obter e excluir

obter rejeita IDs não positivos e converte Optional.empty em NaoEncontradoException. excluir rejeita IDs não positivos e também a ausência de linha afetada. O serviço pode ser utilizado fora do HTTP sem precisar inventar um código de resposta; ele expressa o significado e deixa a tradução para a camada de controle.

8.5 texto e limpar

texto centraliza obrigatoriedade e tamanho máximo, recebendo chave, descrição amigável e limite. limpar garante a normalização consistente e evita NullPointerException para campos não enviados. String.length mede unidades UTF-16, o mesmo modelo básico utilizado pelos limites HTML do formulário; não é apresentado como uma contagem linguística de grafemas.

9. Montagem das dependências

AplicacaoListener é um ServletContextListener. Na inicialização do contexto, utiliza uma fábrica previamente configurada para testes ou cria ConexaoJdbc.doAmbiente. Inicializa a estrutura e constrói JdbcItemMidiaDao, ItemMidiaService e o atributo de contexto itemService.

Esse ponto é chamado de composição da aplicação porque reúne os objetos e suas dependências. A Servlet recupera o serviço em init. A instância do serviço é compartilhada, mas não guarda dados de formulário ou de usuário; esses dados são variáveis locais por chamada. O DAO também mantém apenas a fábrica. Os recursos JDBC são locais a cada operação.

10. SegurancaFilter

doFilter converte os objetos de entrada para os contratos HTTP e define UTF-8 antes da leitura de parâmetros. Depois adiciona cabeçalhos de segurança e cache. A política de conteúdo aceita recursos do próprio site, impede enquadramento e restringe a submissão de formulários à origem atual.

O filtro obtém a sessão. Dentro de synchronized(sessao), procura um token existente ou gera 32 bytes com SecureRandom. HexFormat transforma esses bytes em 64 dígitos hexadecimais. A sincronização evita inicializações concorrentes inconsistentes do token da mesma sessão.

Em POST, lê csrfToken do formulário e compara com MessageDigest.isEqual. Token ausente ou diferente produz 403 e a JSP de erro, sem executar a cadeia restante. Se for válido, chain.doFilter permite que a Servlet prossiga. Em GET não há comparação de token; o token é preparado para os formulários.

O token não substitui login, autorização, HTTPS ou política de cookies. Sua função específica é conferir que uma escrita veio de um formulário associado à sessão corrente. Os testes verificam ausência e incorreção do token.

11. ItemMidiaServlet

11.1 init e estado

init recupera o serviço do contexto. Se ele estiver ausente, a inicialização falha claramente. O único estado de aplicação da Servlet é essa referência; campos de requisição não são armazenados em atributos da Servlet, porque uma mesma instância pode atender requisições simultâneas.

11.2 doGet

Carrega e remove a mensagem flash da sessão, tornando-a disponível apenas para a próxima visualização. O switch decide pela rota: / lista e busca; /novo prepara cadastro; /editar carrega dados existentes; /detalhes apresenta a obra. /salvar e /excluir retornam 405 com Allow: POST. Rotas desconhecidas retornam 404.

req.setAttribute cria dados válidos durante a requisição. exibir usa forward para a JSP protegida. O navegador recebe HTML; ele não acessa diretamente WEB-INF, nem vê o código Java da página.

11.3 doPost

Na rota salvar, lê ID opcional e os seis campos. Chama servico.salvar. Em sucesso, grava uma mensagem flash e retorna 303 com Location para os detalhes. Na falha de validação, mantém o mapa original, associa ID e erros, prepara opções e responde 422 com a JSP de formulário.

Na rota excluir, chama servico.excluir, registra confirmação e retorna 303 para a lista. Rotas POST não reconhecidas retornam 404. As requisições chegam aqui apenas depois de o filtro conferir o token.

11.4 executar

Recebe uma ação cujo corpo corresponde a GET ou POST. Concentra a tradução de exceções previsíveis: item ausente para 404; número inválido para 400; validação de busca para 400; persistência para 503 e log. IOException e ServletException permanecem no contrato do contêiner, pois não são convertidas silenciosamente em sucesso.

11.5 id e rota

id exige o padrão [1-9][0-9]* e utiliza Long.parseLong. Isso rejeita zero, sinal, letras, ausência e overflow. rota transforma pathInfo ausente em /. A aplicação tem um único mapeamento /itens/* e não usa o parâmetro de ação para escolher uma classe arbitrária.

11.6 lerCampos e prepararFormulario

lerCampos aceita somente as chaves previstas. Valores ausentes viram texto vazio para que o serviço produza mensagens de obrigatoriedade. prepararFormulario transforma uma entidade em mapa de textos, ou define LIVRO como opção inicial para um cadastro. O mesmo formulario.jsp serve ao cadastro e à edição.

11.7 opcoes, exibir, erro e redirecionar

opcoes disponibiliza enum e ano máximo à interface. O atributo HTML max melhora a interação, mas a regra é conferida novamente no serviço. exibir monta apenas nomes de JSP constantes escolhidos pelo programa. erro define o status e a mensagem antes de encaminhar. redirecionar utiliza o contextPath para funcionar tanto na raiz local quanto em /catalogo no deploy WAR.

O código 303 orienta o navegador a fazer um GET após o POST. Ao atualizar a tela de detalhes, o navegador não repete o cadastro. Se alguém envia deliberadamente dois POSTs de cadastro, ainda pode criar duas obras; não há uma chave de idempotência no escopo entregue.

12. ServidorLocal e web.xml

ServidorLocal.iniciar prepara o diretório Tomcat, define a porta e o endereço 127.0.0.1, limita o corpo POST a 64 KiB e adiciona o webapp. O classloader pai permite localizar as classes e dependências do processo. O parâmetro FabricaConexao permite executar o mesmo webapp contra um banco isolado nos testes.

main lê CATALOGO_PORT, inicia target/catalogo e registra um shutdown hook para encerrar o Tomcat. O servidor fica aguardando no terminal. Nos testes, a porta 0 solicita uma porta livre ao sistema operacional, evitando fixar 8080 e colidir com outras aplicações.

web.xml registra listener, filtro e Servlet explicitamente. A ordem de inicialização disponibiliza o serviço antes de carregar a Servlet. Os mapeamentos /itens/* e /* organizam roteamento e filtro. A sessão expira em 30 minutos, o cookie é HttpOnly e o rastreamento é apenas por cookie. As páginas de erro 404 e 500 encaminham para a JSP de erro protegida. Um Tomcat externo usa esse mesmo descritor sem precisar executar ServidorLocal.main.

13. Páginas JSP e recursos do navegador

13.1 tags.jspf, cabecalho.jspf e rodape.jspf

Os fragmentos evitam repetir declarações, HTML comum, navegação e rodapé. tags.jspf declara as bibliotecas c e fn antes da primeira utilização de uma tag em cada JSP. O título da página é exibido por c:out, inclusive quando vem de um título digitado pelo usuário. Os caminhos de CSS, JavaScript e links usam contextPath.

Há link de salto para o conteúdo, navegação identificada e atributo lang=pt-BR. Cada fragmento que contém texto acentuado declara pageEncoding UTF-8 para que a leitura do arquivo incluído não gere caracteres incorretos. O rodapé fecha main, body e html. São inclusões estáticas de apresentação, sem consulta ao banco ou validação de negócio.

13.2 lista.jsp

c:choose diferencia coleção vazia e coleção com itens. c:forEach percorre request.itens e cria os cartões. fn:length calcula o total encontrado. A busca usa GET para que seu endereço possa ser reutilizado. O valor reapresentado no atributo de input utiliza fn:escapeXml.

Título, autor, gênero e sinopse são apresentados por c:out. O tipo usa a constante do enum na classe CSS e a descrição legível no texto. ID e ano são valores numéricos. A arte tipográfica de cada cartão deriva da primeira letra do título, escapada antes de entrar no HTML.

13.3 formulario.jsp

O ID determina título, texto do botão e inclusão do campo oculto de edição. O token de sessão acompanha o POST. Os campos usam required, maxlength e limites de ano para melhorar o preenchimento; o servidor não confia somente nessas validações do navegador.

Os valores dos atributos HTML são escapados por fn:escapeXml; o conteúdo de textarea é c:out. Isso inclui o formulário reapresentado após um erro, um ponto importante porque a entrada ainda não foi gravada. As mensagens por campo estão associadas por aria-describedby. O resumo de erros utiliza role=alert.

13.4 detalhes.jsp

Apresenta a obra inteira, metadados e sinopse preservando quebras de linha com CSS. O botão de exclusão abre um details que contém a confirmação e o formulário POST. Sem JavaScript, o usuário pode abrir o summary normalmente. Não há um link GET que apague a obra.

13.5 erro.jsp e index.jsp

erro.jsp apresenta uma mensagem pública ou um texto padrão, nunca o conteúdo bruto de uma exceção interna. index.jsp redireciona a raiz para /itens. Esse redirecionamento técnico pequeno é o único scriptlet; as JSPs de negócio usam EL/JSTL e não incluem SQL ou chamadas ao DAO.

13.6 style.css

As variáveis em :root definem a paleta. container centraliza o conteúdo. card-grid utiliza três colunas no desktop, duas em telas intermediárias e uma em telas pequenas. Os formulários e detalhes também adaptam a disposição. Estilos de foco e contraste foram previstos; ainda é necessária uma avaliação específica para afirmar conformidade formal de acessibilidade.

As capas usam cores, letras e formas CSS, evitando dependências de imagens externas. A classe art-LIVRO, art-FILME ou art-SERIE seleciona a paleta por categoria. prefers-reduced-motion reduz transições para quem solicita menos animação.

13.7 app.js

Atualiza o contador da sinopse no evento input, sem alterar o texto enviado. O botão de exclusão apenas abre o painel, aproxima-o da área visível e posiciona o foco na confirmação. O script utiliza textContent para atualizar texto, não innerHTML. O CRUD não depende de JavaScript para ser executado.

14. Fluxos completos para depuração

14.1 Cadastro válido

GET novo → token de sessão → formulario.jsp → POST salvar → filtro confere token → Servlet lê campos → serviço valida → DAO INSERT → H2 gera ID → DAO devolve entidade → 303 → GET detalhes → SELECT por ID → detalhes.jsp. Se algum elo falhar, inspecione a camada responsável antes de alterar o restante.

14.2 Cadastro inválido

POST com token válido → serviço acumula erros → nenhuma escrita DAO → ValidacaoException → Servlet define status 422 e atributos form/erros → JSP reapresenta o formulário. A validação não encerra o servidor e não perde as entradas digitadas.

14.3 Consulta sem resultado e falha SQL

ID existente em formato correto, mas ausente no banco → Optional.empty → NaoEncontradoException → 404. Banco inacessível → SQLException no DAO → PersistenciaException com causa → log e 503. Esses dois caminhos representam problemas diferentes e não são confundidos com uma lista vazia.

15. Testes e o que cada grupo comprova

ItemMidiaDaoEmMemoria guarda itens em LinkedHashMap e simula IDs. É um dublê de teste, não o armazenamento da execução normal. ItemMidiaServiceTest usa esse dublê e Clock fixo para testar normalização, erros acumulados, limites exatos e inválidos, atualização, exclusão e termo de busca. O contador de escritas demonstra que entradas inválidas não chegam à persistência.

JdbcItemMidiaDaoTest usa bancos H2 em memória com nomes UUID, além de um banco em arquivo numa pasta temporária. Verifica todos os campos após UPDATE, SQL Injection literal, busca com aspas e curingas, restrições do banco, reabertura do arquivo e inicialização que não apaga dados.

CatalogoHttpIT inicia Tomcat real, utiliza HttpClient com CookieManager, lê o token do formulário e envia POSTs codificados. Verifica CRUD completo, JSPs de cadastro e edição, preservação de valores, códigos de falha, CSRF, XSS nas três telas, busca maliciosa, cabeçalhos e indisponibilidade do banco. A falha simulada do banco ocorre depois da inicialização e é restaurada ao fim do teste.

O relatório VERIFICACAO.md é produzido a partir dos XMLs JUnit; o número de testes parametrizados considera cada combinação executada. Não se converte uma lista de testes planejados em resultados executados. Os testes HTTP verificam HTML e comportamento; as capturas de navegador permitem conferir a apresentação visual.

16. Documentação, evidências e empacotamento

scripts/gerar_documentos.py transforma os arquivos Markdown em Word e PDF, incorpora diagramas e telas, gera o relatório factual de testes e reúne o ZIP. As bibliotecas desse script estão listadas em scripts/requirements-docs.txt. Elas são ferramentas editoriais e não fazem parte do runtime Java.

docs/diagramas contém fontes Mermaid e desenhos SVG/PNG para leitura nos documentos. docs/telas contém capturas reais de um servidor de demonstração, cujo banco é separado da pasta data. O utilitário dados_demonstracao.py submete os formulários HTTP e recusa popular um catálogo que já possua itens.

A pasta entrega reúne os documentos, o WAR e um pacote de código com fontes e testes. Caches de Maven, ferramentas baixadas, dados pessoais, .git e diretórios temporários são excluídos. O pacote inclui target/classes, target/dependency e target/catalogo já preparados para iniciar com iniciar.ps1 -SemCompilar durante a apresentação; o código-fonte continua sendo a referência para alterações.

17. Como fazer alterações sem desfazer a separação

Para adicionar um campo, altere o esquema e planeje uma migração para bancos existentes; apenas IF NOT EXISTS não adiciona colunas. Depois atualize entidade, mapeamento/parâmetros DAO, validação, formulário e testes. Essa mudança atravessa camadas porque modifica o modelo de dados, mas cada responsabilidade continua localizada.

Para mudar a regra de ano, altere o serviço, o limite apresentado no formulário e os testes de fronteira. Para mudar o visual, altere JSP/CSS sem criar dependência da persistência na apresentação. Para outro armazenamento, implemente ItemMidiaDao e modifique a composição, testando a equivalência do contrato. Para várias escritas atômicas, projete uma fronteira transacional explícita antes de simplesmente chamar vários DAOs em autocommit.

Os próximos recursos devem responder a necessidades concretas. Login exige autenticação e autorização; paginação exige parâmetros de página e consultas limitadas; edição simultânea exige controle de versão. A arquitetura facilita localizar esses trabalhos, mas não os implementa automaticamente.